Project - Low Level Design

on

Dockerized Entertainment React Service

Course Name: Devops

Institution Name: Medicaps University

Course: Datagami Skill Based Course

Student Name(s) & Enrollment Number(s):

Sr no	Student Name	Enrolment Number
1	PRABAL SHUKLA	EN22CS301713
2	PRATIK KOUSHAL	EN22CS301747
3	PRIYANSH CHOUHAN	EN22CS301759
4	PUSHPRAJ SINGH CHOUHAN	EN22CS301774
5	RAJAT MALVIYA	EN22CS301783

Group : 11 D7

Project no:DO-34

Industry Mentor Name: Aashruti Shah

University Mentor Name: Shyam Patel

Academic Year:2025-2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. References

1. Introduction :

1.1 Scope of Document

This document presents the **Low-Level Design (LLD)** of the project titled **"Dockerized Entertainment React Service"**. The purpose of this document is to describe the internal working, design decisions, and implementation details of the system.

The scope of this document includes:

- Detailed explanation of the React-based frontend application
- Description of Docker-based containerization process
- Step-by-step explanation of Dockerfile instructions
- Representation of system flow using UML diagrams
- Explanation of system components and their interactions

The document ensures that the system design is clearly understood and can be easily implemented, maintained, and extended in the future.

1.2 Intended Audience

This document is intended for:

- **Developers** : To understand application structure and code-level design
- **DevOps Engineers** : To understand containerization and deployment workflow
- **Project Evaluators / Faculty** :To evaluate system design and implementation
- **Future Contributors** : To extend the system with backend services and CI/CD pipelines

1.3 System Overview

The system is a **web-based entertainment application** developed using React.js. It provides a simple user interface displaying a list of movies along with their ratings.

The application is containerized using Docker, which provides:

- **Portability** : Runs on any system with Docker installed

- **Consistency** : Same behavior across environments
- **Isolation** : Dependencies are isolated within the container

Currently, the system is a **frontend-only application**, but it is designed in such a way that it can be extended in the future with backend APIs, databases, CI/CD pipelines, and Kubernetes deployment.

2. System Design :

2.1 Application Design

The application follows a layered architecture that separates user interaction, application logic, and deployment environment.

User → Browser → Docker Container → React Application

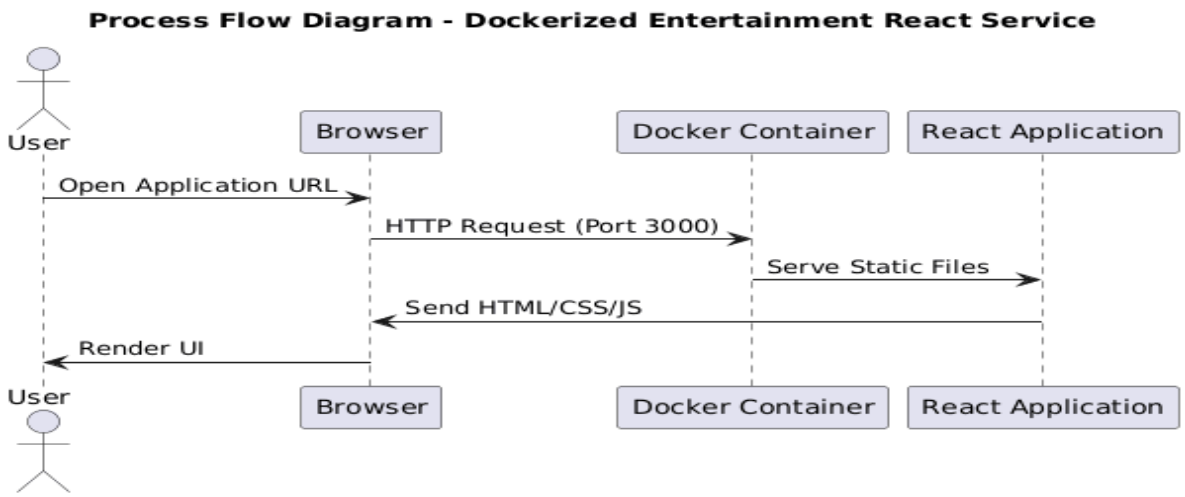
Detailed Explanation:

- The **user interacts** with the system using a web browser
- The browser sends an HTTP request to the Docker container
- The Docker container hosts the built React application
- The React application processes the request and returns UI content
- The browser renders the UI for the user

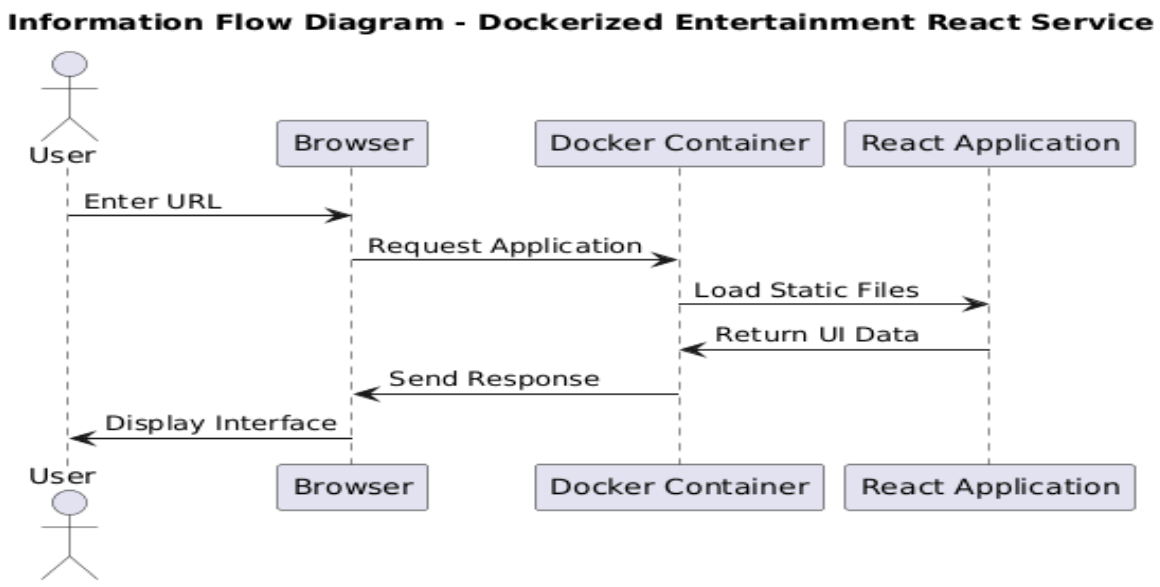
This design ensures modularity and separation of concerns.

2.2 Process Flow

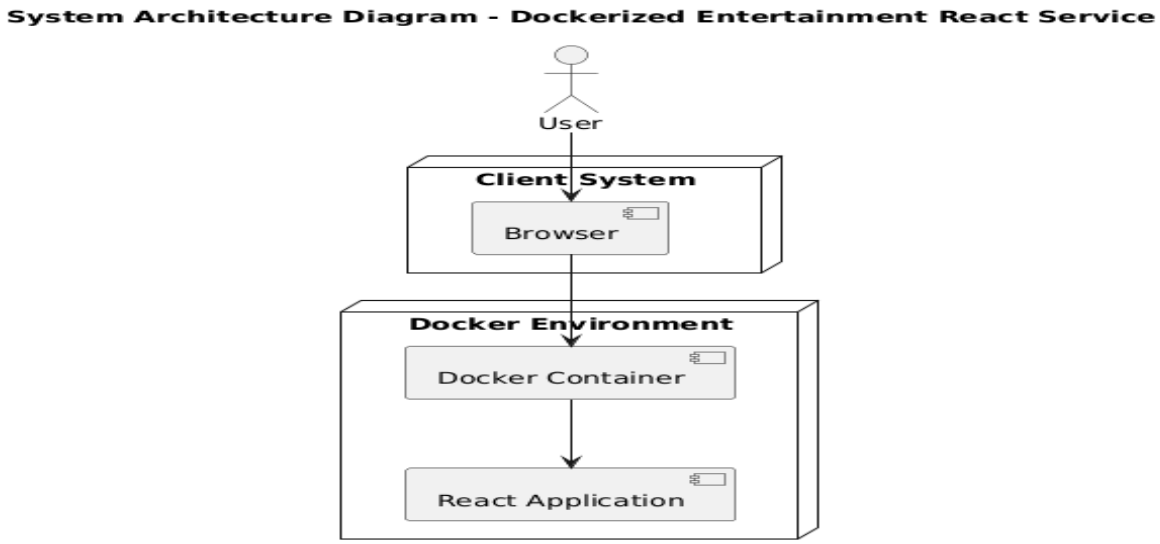
Description: The process flow represents the sequence of operations performed when a user accesses the application.



2.3 Information Flow : The information flow describes how data is transferred between different components of the system.



2.4 System Architecture : The architecture diagram provides a high-level representation of system components and their relationships.



2.5 Components Design

The system consists of multiple components working together

1. React Frontend

- Developed using React and Vite
- Responsible for UI rendering

- Displays movie titles and ratings
- Lightweight and fast

2. Dockerfile

- Defines instructions to build the Docker image
- Specifies base image, dependencies, and commands
- Ensures consistent build process

3. Docker Image

- Packaged version of the application
- Includes all dependencies and build files
- Portable across environments

4. Docker Container

- Runtime instance of Docker image
- Executes the application
- Exposes port for user access

5. Browser

- Acts as client interface
- Sends requests and displays responses

2.6 Key Design Considerations

1. Simplicity : The UI is kept simple to focus on DevOps implementation rather than frontend complexity.

2. Portability : Docker ensures the application runs consistently across systems.

3. Scalability : The system can be extended with backend services and Kubernetes deployment.

4. Maintainability : Clean and minimal code structure makes the system easy to maintain.

5. Performance : Static build ensures faster loading and efficient resource usage.

6. Extensibility : Future enhancements like APIs, database, and CI/CD can be easily integrated.

2.7 API Catalogue

Current Implementation:

No APIs available (Static frontend application for now)

3. References :

- Docker Official Documentation
- React.js Documentation
- Vite Documentation
- DevOps Best Practices